

Contents

1	Lab Chapter 1: The Research Computing Environment	2
1.1	Chapter Opening	2
1.2	Prerequisites Check	3
1.3	Environment Setup	4
1.3.1	Step 1: Choose conda or venv	4
1.3.2	Step 2: Create the repository folder	4
1.3.3	Step 3: Create the Python environment with conda	4
1.3.4	Step 4: venv fallback	5
1.3.5	Step 5: Verify the Python scientific stack	6
1.3.6	Step 6: Register the Jupyter kernel	6
1.3.7	Step 7: Install or verify MATLAB	6
1.3.8	Step 8: Install and verify Git	7
1.3.9	Step 9: Minimal environment hello world	7
1.3.10	Common Install Failures and Fixes	8
1.4	Conceptual Bridge: From Battery Theory to Reproducible Research Objects	8
1.5	Guided Walkthrough 1: Build and Freeze the Python Environment	10
1.5.1	Walkthrough 1 code	10
1.5.2	Walkthrough 1 code explanation	11
1.5.3	Walkthrough 1 expected output	12
1.5.4	Walkthrough 1 troubleshooting	12
1.5.5	Walkthrough 1 reflection	13
1.6	Guided Walkthrough 2: Create a Research Project Scaffold That Will Still Make Sense Later	13
1.6.1	Walkthrough 2 code	13
1.6.2	Walkthrough 2 code explanation	15
1.6.3	Walkthrough 2 expected output	15
1.6.4	What correct structure looks like	15
1.6.5	Walkthrough 2 troubleshooting	16
1.6.6	Walkthrough 2 reflection	16
1.7	Guided Walkthrough 3: Start a Research Log and Capture Decisions Before They Disappear	16
1.7.1	Walkthrough 3 code	16
1.7.2	Suggested first log entry content	18
1.7.3	Walkthrough 3 code explanation	19
1.7.4	Walkthrough 3 expected output	19
1.7.5	What belongs in the log and what does not	19
1.7.6	Walkthrough 3 troubleshooting	20
1.7.7	Walkthrough 3 reflection	20
1.8	Guided Walkthrough 4: Initialize Git and Build a Commit History Worth Keeping	20
1.8.1	Walkthrough 4 code	20
1.8.2	A meaningful 10-commit sequence	22
1.8.3	Walkthrough 4 code explanation	22
1.8.4	Walkthrough 4 expected output	22

1.8.5	What makes a commit “meaningful” in research	23
1.8.6	Walkthrough 4 troubleshooting	23
1.8.7	Walkthrough 4 reflection	23
1.9	Guided Walkthrough 5: Run a “Hello Battery” Sanity Check in Python and MATLAB	23
1.9.1	Part A: Python sanity check	24
1.9.2	Part B: MATLAB toolbox verification and sanity check	27
1.10	Open-Ended Exercises	31
1.10.1	Exercise 1	31
1.10.2	Exercise 2	31
1.10.3	Exercise 3	31
1.11	What Changes for Sodium-Ion?	31
1.12	Worked Solutions to the Open-Ended Exercises	32
1.12.1	Solution to Exercise 1	32
1.12.2	Solution to Exercise 2	33
1.12.3	Solution to Exercise 3	34
1.13	Chapter Summary and Skill Checklist	35
1.14	Deliverable	36
1.15	Further Practice and Reading	37

1 Lab Chapter 1: The Research Computing Environment

1.1 Chapter Opening

This chapter is about a skill that sounds administrative until the day it saves your paper: building a research computing environment you can trust. By the end of the chapter, you will not merely have Python, Jupyter, MATLAB, and Git installed. You will have a working repository, an isolated environment, a repeatable project layout, a research log template, and a pair of sanity-check simulations that prove your tools are actually behaving the way you think they are. That combination is what separates “I got a plot once” from “I can still reproduce this result six weeks later, explain where it came from, and defend it in front of an advisor or reviewer.”

This matters because simulation research fails in boring ways long before it fails in interesting ways. A paper is not weakened only by a wrong diffusion coefficient or a bad thermal assumption. It is also weakened by a missing package version, a notebook that only runs on one laptop, an unlabeled CSV, a folder full of `final_results_v2_really_final`, or a figure whose exact generation path has been forgotten. Reviewers rarely see those internal failures directly, but they feel them in the form of irreproducible methods, vague parameter provenance, suspiciously convenient plots, and “we were unable to verify your claimed workflow” when someone in the group tries to build on your work later. The point of this chapter is to prevent those failures before they become habits.

This companion is a methods book, so we will treat environment setup as part of battery research rather than a preface to it. Later chapters will ask you to run PyBaMM models, identify equivalent-circuit parameters from public datasets, implement Kalman filters in MATLAB, and reproduce published figures. None of that work is credible if you cannot say exactly which software versions you used, which script

generated which output, which assumptions were teaching shortcuts and which were field-standard practice, and where each intermediate result was stored. Research competence is not only the ability to derive equations from first principles. It is also the ability to preserve the chain from equation to executable model to stored artifact.

Keep several theory-textbook chapters nearby while you work. Textbook Chapter 3, *Performance Metrics and Terminology*, is the one to keep most open during this chapter, because every file name, axis label, unit annotation, and sanity-check output should use the vocabulary from that chapter consistently. When we record current, voltage, energy, capacity, SOC, and C-rate in files and plots, we are operationalizing that chapter. Textbook Chapter 7, *Degradation Mechanisms*, matters here too, because degradation claims are especially vulnerable to poor provenance: if you do not know which dataset, temperature window, and preprocessing code produced a trend, you do not know what physical mechanism you are actually looking at. Textbook Chapter 8, *Heat in Batteries*, will become relevant as soon as we start running electrothermal models; the same reproducibility discipline we build here will later protect you from silently mixing ambient temperature assumptions or heat-generation conventions. Textbook Chapter 10, *State Estimation*, is also worth keeping in mind because BMS algorithms look mathematically neat on paper and become fragile as soon as data conventions drift. This chapter is the infrastructure layer underneath all of them.

One more framing point matters for this manual in particular: you do not have wet-lab access. That is not a weakness if you are deliberate. It means your research identity will be built around careful simulation, disciplined use of public datasets, strong reproduction practice, and clean software methods. In other words, your credibility will come from the exact things we are building here. A simulation-based battery researcher can produce publishable work without touching an Arbin cycler, but not without knowing how to keep environments isolated, analyses versioned, and outputs traceable.

So we will move slowly and do the unglamorous work properly. We will decide when to use conda and when to use venv. We will pin package versions instead of trusting memory. We will create a project structure that still makes sense when the project doubles in size. We will verify that Jupyter sees the correct kernel. We will check MATLAB installation status for the toolboxes we need later. We will initialize Git before the repository is messy. We will create a research log template before there is anything exciting to log. And then we will end with a “hello battery” in both Python and MATLAB, because a battery methods workflow should prove itself on battery-flavored output, not just `print("hello world")`.

If you do this chapter carefully, later chapters become much easier. If you rush it, later chapters become haunted. Let’s build the boring foundation well enough that the interesting work can stand on it.

1.2 Prerequisites Check

- Required software: Miniforge or Anaconda for environment management; Python 3.11; Git 2.43+; JupyterLab 4.4+; MATLAB R2025b recommended, R2024b acceptable

- Python packages used in this chapter: `numpy==2.3.4`, `scipy==1.16.0`, `pandas==3.0.2`, `matplotlib==3.10.8`, `jupyterlab==4.4.10`, `ipykernel==7.2.0`
- MATLAB products to verify: Simulink, Simscape, Simscape Battery, Control System Toolbox, Optimization Toolbox
- Required textbook chapters: Textbook Chapter 3 is essential; Chapters 7, 8, and 10 should be familiar conceptually
- Required prior lab chapters: none
- Estimated time: 6 to 8 hours if installs go smoothly; 8 to 10 hours if you are also installing MATLAB or learning Git from scratch

If your Python fundamentals are shaky, pause and review array creation, scripts vs notebooks, and package imports before continuing. If Git is entirely new, that is fine; this chapter assumes no research-Git experience and builds only what we need. If MATLAB is not installed yet, do not skip the verification section. Later chapters in Part III will depend on it.

1.3 Environment Setup

We will set up the project in a way that is reproducible, boring, and strong. That is exactly the right combination.

1.3.1 Step 1: Choose conda or venv

For this manual, conda is the default recommendation and venv is the fallback. The reason is practical rather than ideological. Battery research stacks often include scientific libraries with compiled dependencies, and conda-forge resolves those dependencies more smoothly across Windows, macOS, and Linux than plain pip does.

Use this rule:

- **conda**: use it when you want the least painful scientific-stack install. Tradeoff: a slightly heavier environment manager.
- **venv**: use it when you already have a clean system Python and want minimal tooling. Tradeoff: more likely to hit platform-specific wheel issues.

If you do not already have a strong preference, use conda.

1.3.2 Step 2: Create the repository folder

Create a working folder anywhere you keep research projects. I will call it `sib-research-companion`, but you may choose another name.

```
mkdir sib-research-companion
cd sib-research-companion
```

The expected result is simple: your terminal prompt should now show that you are inside the new folder. If you run `pwd` on Linux or macOS, or `cd` on Windows PowerShell, you should see the new path.

1.3.3 Step 3: Create the Python environment with conda

Create a file named `environment.yml` in the repository root with the following contents.

```

name: sib-research
channels:
  - conda-forge
dependencies:
  - python=3.11
  - numpy=2.3.4
  - scipy=1.16.0
  - pandas=3.0.2
  - matplotlib=3.10.8
  - jupyterlab=4.4.10
  - ipykernel=7.2.0
  - pip

```

This file is intentionally small. It is the human-maintained description of the environment, not the machine-generated lockfile. We pin the major tools exactly because this manual is teaching research habits, not “install whatever was current on the day you happened to run the notebook.”

Now create and activate the environment:

```

conda env create -f environment.yml
conda activate sib-research

```

If `conda activate` fails because your shell has not been initialized, run the shell hook once:

```
conda init bash
```

Then close and reopen the terminal and activate again.

If you prefer `mamba`, the command is equivalent and often faster:

```

mamba env create -f environment.yml
mamba activate sib-research

```

1.3.4 Step 4: venv fallback

If you are not using `conda`, create a `requirements-venv.txt` file:

```

numpy==2.3.4
scipy==1.16.0
pandas==3.0.2
matplotlib==3.10.8
jupyterlab==4.4.10
ipykernel==7.2.0

```

Then create and activate the environment.

On Linux or macOS:

```

python3.11 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements-venv.txt

```

On Windows PowerShell:

```

py -3.11 -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements-venv.txt

```

The `pip install --upgrade pip` line matters. Many beginner installation failures on Windows trace back to using an old pip against a modern wheel ecosystem.

1.3.5 Step 5: Verify the Python scientific stack

Run this exact verification snippet from the repository root after activating the environment.

```
python - <<'PY'
import numpy as np
import scipy
import pandas as pd
import matplotlib

print("Scientific Python stack verified.")
print(f"NumPy      : {np.__version__}")
print(f"SciPy      : {scipy.__version__}")
print(f"Pandas     : {pd.__version__}")
print(f"Matplotlib  : {matplotlib.__version__}")
PY
```

You should see terminal output of this form:

```
Scientific Python stack verified.
NumPy      : 2.3.4
SciPy      : 1.16.0
Pandas     : 3.0.2
Matplotlib : 3.10.8
```

The exact patch versions should match your pins. If they do not, you are not in the environment you think you are in.

1.3.6 Step 6: Register the Jupyter kernel

A common mistake is installing packages into one environment and launching Jupyter from another. We prevent that now.

```
python -m ipykernel install --user --name sib-research --display-name "Python
↪ (sib-research)"
```

Launch JupyterLab:

```
jupyter lab
```

Expected behavior: a browser tab opens or Jupyter prints a local URL that you can paste into a browser. In the launcher, the kernel list should contain Python (sib-research). Create a new notebook with that kernel and run:

```
import sys
print(sys.executable)
```

The path should point into your sib-research environment, not system Python.

1.3.7 Step 7: Install or verify MATLAB

For the MATLAB side of this manual, MATLAB R2025b is the recommended target version. R2024b is acceptable if that is what your license or trial provides. Later chapters will lean on products beyond base MATLAB, so install or verify these now:

- MATLAB

- Simulink
- Simscape
- Simscape Battery
- Control System Toolbox
- Optimization Toolbox

Use the MathWorks installer and choose a custom install if you have the option, so you can explicitly select toolboxes. If you are using a trial or campus license, availability of Simscape Battery is the item to check most carefully; base MATLAB alone will not be enough for Part III.

Open MATLAB and run:

```
ver
```

You should see a version table listing installed products. Do not worry yet if the list is long. In the guided walkthrough below, we will write a script that checks only the products we care about.

1.3.8 Step 8: Install and verify Git

Check Git:

```
git --version
```

You should see something like:

```
git version 2.43.0
```

Now configure your identity once:

```
git config --global user.name "Your Name"
git config --global user.email "your_email@example.com"
git config --global init.defaultBranch main
```

These settings become part of your commit metadata. If the email is wrong, your GitHub history later becomes harder to trace correctly.

1.3.9 Step 9: Minimal environment hello world

Before we build the battery-specific sanity check later, verify that the environment can run a tiny end-to-end Python snippet.

```
python - <<'PY'
import numpy as np
time_s = np.arange(0, 6)
voltage_v = 3.2 + 0.1 * np.cos(time_s)
print("time_s:", time_s.tolist())
print("voltage_v:", np.round(voltage_v, 3).tolist())
PY
```

Expected output:

```
time_s: [0, 1, 2, 3, 4, 5]
voltage_v: [3.3, 3.254, 3.158, 3.101, 3.135, 3.228]
```

The exact floating-point rounding may differ at the third decimal place, but the structure should be identical.

1.3.10 Common Install Failures and Fixes

The most common environment problems are not mysterious. They are usually one of four things.

1. `conda`: command not found
 Symptom: your shell cannot see `conda` at all.
 Fix: install Miniforge or Anaconda, then run `conda init bash` and restart the terminal.
2. `ModuleNotFoundError` after a successful install
 Symptom: `pip` or `conda` said the package installed, but Python still cannot import it.
 Fix: you are probably in the wrong interpreter. Run `which python` on Linux/macOS or `Get-Command python` on PowerShell, and confirm it points to your intended environment.
3. Jupyter launches but the wrong packages appear inside the notebook
 Symptom: the notebook imports fail even though the terminal imports succeed.
 Fix: register the kernel with `python -m ipykernel install --user --name sib-research ...` and then explicitly choose Python (`sib-research`) in JupyterLab.
4. MATLAB starts but needed toolboxes are missing
 Symptom: `ver` does not list Simulink, Simscape, or Simscape Battery.
 Fix: rerun the installer with toolbox selection enabled, or confirm your license includes them. If you only have base MATLAB today, you can still complete this chapter, but Part III will need either a trial or a fuller license.

1.4 Conceptual Bridge: From Battery Theory to Reproducible Research Objects

In the theory textbook, a battery model lives as mathematics, physics, and interpretation. You learned to think in terms of half-reactions, transport limitations, OCV curves, degradation mechanisms, thermal balances, and state estimators. That level is essential. But a research workflow needs one more translation layer: how those ideas become stable computational objects that can be rerun, compared, and defended.

The most important shift is this: a simulation result is not just a curve. It is a claim with dependencies.

Suppose later in this manual you run a PyBaMM DFN model and obtain a terminal-voltage trace that seems to match a published figure. In theory language, that trace depends on equations, boundary conditions, parameters, and a current profile. In research-computing language, it also depends on a specific package version, a specific parameter file, a specific script, and a specific output directory. If any one of those is ambiguous, the result is weaker than it looks. The curve may be visually correct and still be scientifically fragile.

That is why environment management belongs in a battery methods book. The environment is where the abstract model becomes executable. Textbook Chapter 3 taught you to distinguish capacity from energy, OCV from terminal voltage, and SOC from SOH. Here we add another distinction that matters just as much in

practice: the difference between a result and a reproducible result. A result is “I ran something and got 3.27 V after 1800 s.” A reproducible result is “Script `src/hello_battery.py`, run under environment `sib-research`, using `numpy 2.3.4` and `matplotlib 3.10.8`, produced `results/hello_battery_python.csv` and `figures/hello_battery_python.png` from commit `abc1234` on 2026-04-14.” The second form looks fussier, but it is the one you can build a paper or thesis on.

This is also where notebook culture often misleads beginners. Jupyter notebooks are excellent for exploration. They are poor as the sole home of a research workflow. Why? Because notebooks interleave code, output, and state in a way that encourages hidden dependencies. A cell may run only because an earlier cell created a variable yesterday. A figure may be regenerated from a stale in-memory DataFrame rather than from the raw data. For that reason, this manual will keep using notebooks, but we will pair them with scripts in `src/` and with saved outputs in `results/`. That separation is standard practice. Exploratory work can be notebook-first. Reproducible work should become script-backed.

The folder structure matters for the same reason. When you put raw downloads, cleaned datasets, notebooks, reusable code, generated figures, and paper notes into the same directory, you erase provenance. Later, when you revisit a model, you cannot tell whether a CSV is raw, processed, smoothed, manually edited, or exported from MATLAB. A clean project structure is not aesthetic minimalism. It is a physical embodiment of method.

Git plays an equally important role. Many people first meet Git as a collaboration tool, but in research it is just as valuable as a memory tool. A commit is a checkpoint in your reasoning. It lets you say, “This is the exact state of the repository before I changed the preprocessing,” or “This figure was generated before I changed the OCV smoothing routine.” That matters enormously once you start reproducing published papers. If a reproduction attempt diverges from the target figure, you want the freedom to compare commit states rather than guessing which invisible notebook change caused the drift.

Research logs are the human counterpart to Git history. Git can tell you which files changed. It cannot tell you why you distrusted a data column, why you chose one solver tolerance over another, or why you decided a mismatch with a paper was acceptable. Those decisions must live somewhere readable. A dated Markdown log is a remarkably effective choice because it is plain text, version-controllable, diffable, and searchable. It also trains you to think like a researcher rather than a code runner. You are not merely asking, “Did the script execute?” You are asking, “What assumption did I just make, and will future-me remember it?”

This is especially important for sodium-ion work. Textbook Chapter 13 emphasized that sodium-ion workflows cannot always inherit lithium-ion assumptions unchanged. OCV shapes differ. Dataset availability is worse. Figure digitization is more common because parameter sets and full data releases are less standardized. That means provenance matters even more. In a sparse-data setting, every preprocessing choice carries extra weight.

So the bridge from theory to tools is not only about learning commands. It is about learning how a scientific argument is preserved in software form. Equations become scripts and parameter files. Assumptions become log entries and config choices. Out-

puts become versioned artifacts. Reproducibility becomes the operating system of the whole project. With that frame in place, the next sections will feel less like computer setup and more like the beginning of actual battery research.

1.5 Guided Walkthrough 1: Build and Freeze the Python Environment

Learning objective: Create an isolated Python environment, verify that it is the one you are actually using, and generate a machine-readable record of its package versions.

Before we write any battery-flavored code, we need to establish one discipline that will repeat throughout this companion: every environment should be inspectable. It is not enough to say “I used Python with NumPy and SciPy.” That description is too vague for research. We want a human-readable environment specification and a machine-generated environment record.

1.5.1 Walkthrough 1 code

First, create `environment.yml` exactly as shown earlier if you have not already done so. Then create a script called `src/env_report.py`.

```
from __future__ import annotations

import json
import platform
import subprocess
import sys
from datetime import datetime, timezone
from importlib.metadata import PackageNotFoundError, version
from pathlib import Path

PACKAGES_TO_REPORT = [
    "numpy",
    "scipy",
    "pandas",
    "matplotlib",
    "jupyterlab",
    "ipykernel",
]

def package_version(package_name: str) -> str:
    try:
        return version(package_name)
    except PackageNotFoundError:
        return "NOT_INSTALLED"

def git_commit_hash() -> str:
    try:
        return subprocess.check_output(
            ["git", "rev-parse", "HEAD"],
            text=True,
        ).strip()
    except Exception:
        return "NOT_A_GIT_REPOSITORY"

def main() -> None:
    results_dir = Path("results")
```

```

results_dir.mkdir(parents=True, exist_ok=True)

report = {
    "generated_utc": datetime.now(timezone.utc).isoformat(),
    "python_executable": sys.executable,
    "python_version": platform.python_version(),
    "platform": platform.platform(),
    "git_commit": git_commit_hash(),
    "packages": {
        package_name: package_version(package_name)
        for package_name in PACKAGES_TO_REPORT
    },
}

output_path = results_dir / "environment_report.json"
output_path.write_text(json.dumps(report, indent=2), encoding="utf-8")

print(f"Environment report written to: {output_path}")
print(json.dumps(report, indent=2))
if __name__ == "__main__":
    main()

```

Run it:

```

mkdir -p src results
python src/env_report.py

```

1.5.2 Walkthrough 1 code explanation

The `from __future__ import annotations` line is a small modern-Python convenience that keeps type annotations lightweight. It is not strictly necessary here, but it is a good habit and will appear again later.

We import `json`, `platform`, `subprocess`, and `sys` because we want the report to include not only package versions but also interpreter and operating-system information. That is important because a solver bug can be platform-dependent, and a kernel mismatch often shows up first in `sys.executable`.

`datetime.now(timezone.utc).isoformat()` deliberately uses UTC rather than local time. In collaborative or long-running projects, UTC timestamps reduce ambiguity. If you later compare logs from two machines or time zones, UTC keeps entries aligned.

The `package_version` function wraps `importlib.metadata.version`. That call asks the installed environment for a package version without importing the whole package. This is slightly cleaner than importing every package just to read `__version__`, and it works for most Python distributions.

The `git_commit_hash` function tries to capture the current repository state. If Git has not been initialized yet, it returns `NOT_A_GIT_REPOSITORY` instead of crashing. That is intentional. A good environment-report script should degrade gracefully.

Inside `main`, we create `results/` if it does not already exist. This matters because scripts that assume preexisting folders are brittle. We are teaching robust patterns from the start.

The report dictionary contains five categories of information: generation time, Python executable path, Python version, operating-system platform, Git commit, and selected packages. This is not the maximum information you could capture, but it is enough to answer the core reproducibility question: “What environment produced this output?”

The report is written to `results/environment_report.json`. JSON is a sensible choice because it is plain text, structured, easy to diff, and easy to read back later from Python or MATLAB if needed.

1.5.3 Walkthrough 1 expected output

In the terminal, you should see something like:

Environment report written to: `results/environment_report.json`

```
{
  "generated_utc": "2026-04-14T12:34:56.789012+00:00",
  "python_executable":
↪  "/home/yourname/miniforge3/envs/sib-research/bin/python",
  "python_version": "3.11.11",
  "platform": "Linux-6.8.0-57-generic-x86_64-with-glibc2.39",
  "git_commit": "NOT_A_GIT_REPOSITORY",
  "packages": {
    "numpy": "2.3.4",
    "scipy": "1.16.0",
    "pandas": "3.0.2",
    "matplotlib": "3.10.8",
    "jupyterlab": "4.4.10",
    "ipykernel": "7.2.0"
  }
}
```

Three details matter most.

First, `python_executable` should clearly point into your intended environment. If it points to system Python, stop and fix that before doing anything else.

Second, every package should show the pinned version you intended. If one shows `NOT_INSTALLED`, your environment is incomplete.

Third, `git_commit` will probably say `NOT_A_GIT_REPOSITORY` the first time. That is expected. Later, once Git is initialized, the same script will start recording commit hashes automatically. That change is the first small example of how good tooling compounds.

1.5.4 Walkthrough 1 troubleshooting

1. `ModuleNotFoundError: No module named 'importlib.metadata'`

This can happen only on older Python versions.

Fix: use Python 3.11 as specified.

2. The script writes a report, but the versions are not the pinned ones

Symptom: for example, `numpy` shows a different version from `environment.yml`.

Fix: confirm the activated environment with `python -c "import sys; print(sys.executable)"`.

3. `git_commit` always says `NOT_A_GIT_REPOSITORY`

Symptom: even after you think you initialized Git.

Fix: make sure you are running the script from the repository root, not from some parent or sibling folder.

1.5.5 Walkthrough 1 reflection

This exercise taught the first durable pattern of the manual: every meaningful output should be accompanied by enough environment metadata to explain where it came from. We will reuse this idea later when fitting parameters, exporting PyBaMM runs, and reproducing published figures.

1.6 Guided Walkthrough 2: Create a Research Project Scaffold That Will Still Make Sense Later

Learning objective: Build a folder structure that cleanly separates raw data, processed data, notebooks, reusable code, outputs, and paper materials.

A common beginner failure in computational battery work is building a project around a single notebook and a flat directory. That works for one evening. It fails for a semester. We are going to create a cross-platform project scaffold with Python so that anyone on any operating system can reproduce it exactly.

1.6.1 Walkthrough 2 code

Create `src/bootstrap_project.py`:

```
from __future__ import annotations
```

```
from pathlib import Path
```

```
PROJECT_DIRECTORIES = [
    "data/raw",
    "data/processed",
    "notebooks",
    "src",
    "results",
    "figures",
    "papers",
    "logs",
    "matlab",
]
```

```
README_TEXT = """# Sodium-Ion Battery Research Companion Workspace
```

```
This repository contains the hands-on exercises developed from the battery
↪ tools companion manual.
```

```
## Environment
```

```
Create the main Python environment with:
```

```
~~~bash
```

```
conda env create -f environment.yml
conda activate sib-research
python -m ipykernel install --user --name sib-research --display-name "Python
↳ (sib-research)"
~~~
```

Repository layout

- `data/raw/` stores downloaded datasets exactly as obtained.
- `data/processed/` stores cleaned or transformed datasets.
- `notebooks/` stores exploratory notebooks.
- `src/` stores reusable Python scripts and modules.
- `results/` stores machine-readable outputs such as CSV and JSON files.
- `figures/` stores generated figures ready for reports or papers.
- `papers/` stores paper notes, PDFs, and reproduction plans.
- `logs/` stores dated research-log entries.
- `matlab/` stores MATLAB scripts and Live Scripts.

Reproducibility rule

If an output matters, it should be reproducible from a script, recorded in the
↳ log, and traceable to a commit.
"""

```
GITKEEP_DIRECTORIES = [
    "data/raw",
    "data/processed",
    "notebooks",
    "results",
    "figures",
    "papers",
    "logs",
    "matlab",
]
def main() -> None:
    for directory in PROJECT_DIRECTORIES:
        Path(directory).mkdir(parents=True, exist_ok=True)

    for directory in GITKEEP_DIRECTORIES:
        gitkeep_path = Path(directory) / ".gitkeep"
        gitkeep_path.touch(exist_ok=True)

    readme_path = Path("README.md")
    if not readme_path.exists():
        readme_path.write_text(README_TEXT, encoding="utf-8")

    print("Project scaffold created successfully.")
    for directory in PROJECT_DIRECTORIES:
        print(f" - {directory}")
if __name__ == "__main__":
    main()
```

Run it:

```
python src/bootstrap_project.py
```

Then inspect the directory tree:

```
find . -maxdepth 2 -type d | sort
```

On Windows PowerShell, use:

```
Get-ChildItem -Directory -Recurse -Depth 2 | Select-Object FullName
```

1.6.2 Walkthrough 2 code explanation

The `PROJECT_DIRECTORIES` list is the whole design in one place. This is deliberate. If your project structure lives only in shell history, it becomes hard to audit and hard to reuse. By representing the scaffold as a Python list, we make the structure explicit and version-controllable.

Each folder has a methodological role. `data/raw/` is sacred: files there should be exact downloads from public sources and should not be edited in place. `data/processed/` is where parsed, renamed, resampled, or cleaned files go. `notebooks/` is exploratory. `src/` is reusable code. `results/` and `figures/` hold outputs, but they are separated because machine-readable tables and publication-ready figures are not the same artifact. `papers/` is where reading notes, PDFs, and reproduction plans live. `logs/` captures reasoning. `matlab/` isolates the second language ecosystem we will need later.

The `README_TEXT` string is not filler. A repository without a short explanation of its purpose, layout, and environment commands becomes opaque surprisingly fast. Think of the `README` as the front panel of the lab bench.

The `GITKEEP_DIRECTORIES` list exists because Git does not track empty directories. The `.gitkeep` files are placeholders that let you commit an intended structure before data and outputs arrive.

The script only writes `README.md` if it does not already exist. That protects later manual edits and is a small example of idempotent design: you should be able to rerun setup scripts without damaging work.

1.6.3 Walkthrough 2 expected output

The terminal should print:

```
Project scaffold created successfully.
```

- data/raw
- data/processed
- notebooks
- src
- results
- figures
- papers
- logs
- matlab

When you inspect the directory tree, you should see those folders under the repository root. The exact ordering may differ by platform, but nothing should be missing.

1.6.4 What correct structure looks like

A good structure after this step has three properties.

First, it is semantically meaningful. Someone joining your project should be able to guess where a downloaded dataset belongs and where a generated figure belongs without asking you.

Second, it separates source from artifact. Scripts go in `src/`; generated outputs go in `results/` or `figures/`. This keeps the repository conceptually clean.

Third, it is expandable. When we later add dataset loaders, PyBaMM experiments, MATLAB estimators, and reproduction notebooks, this scaffold will still hold.

1.6.5 Walkthrough 2 troubleshooting

1. The script runs but folders are created in the wrong place

Symptom: you accidentally ran it from your home directory or a parent folder.

Fix: `cd` into the repository root and run it again.

2. The `README.md` does not appear

Symptom: the script created folders but not the `README`.

Fix: check whether a `README` already existed. The script intentionally does not overwrite one.

3. `find` is not available

Symptom: the directory-inspection command fails on Windows or minimal shells.

Fix: use the platform-specific alternative shown above.

1.6.6 Walkthrough 2 reflection

This exercise taught that reproducibility begins with physical organization. Later, when we download CALCE or NASA datasets, or when we export model outputs for MATLAB, the distinction between raw, processed, and generated artifacts will save you from subtle mistakes.

1.7 Guided Walkthrough 3: Start a Research Log and Capture Decisions Before They Disappear

Learning objective: Create a log system that records decisions, failures, assumptions, and next steps in a form that Git can track and future-you can trust.

Most beginners think a research log is for major milestones. In practice, it is more valuable for the small decisions that would otherwise evaporate: why you distrusted a data column, why you clipped a current spike, why you changed a solver, why you think a reproduction mismatch is the paper's fault rather than yours. Git tracks file states. The research log tracks intent and interpretation.

1.7.1 Walkthrough 3 code

Create `src/create_log_entry.py`:

```
from __future__ import annotations
```

```
from datetime import datetime
```

```
from pathlib import Path
```

```
TEMPLATE = """# Research Log: {date}
```


Objective

Write one sentence describing today's concrete objective.

Environment

- Active Python environment:
- MATLAB version:
- Git commit:
- Dataset or paper in use:

Work Performed

Describe what you actually did, not what you planned to do.

Observations

Record numerical outputs, figure behavior, runtime notes, or anything
 ↳ surprising.

Decisions and Rationale

Explain why you chose a solver, parameter bound, preprocessing rule, or file
 ↳ organization choice.

Problems Encountered

Record failures honestly. Include exact error messages where useful.

Next Steps

Write the next one to three actions you would take if you resumed tomorrow.
 """

```
def main() -> None:
    logs_dir = Path("logs")
    logs_dir.mkdir(parents=True, exist_ok=True)

    date_string = datetime.now().strftime("%Y-%m-%d")
    output_path = logs_dir / f"{date_string}_setup.md"

    if output_path.exists():
        print(f"Log entry already exists: {output_path}")
        return

    output_path.write_text(TEMPLATE.format(date=date_string), encoding="utf-8")
    print(f"Created log entry: {output_path}")
if __name__ == "__main__":
    main()
```

Run it:

```
python src/create_log_entry.py
```

Now open the generated file in your editor and replace the placeholders with your real setup notes. A first entry might include package installation success, MATLAB toolbox status, and whether GitHub remote setup is pending.

1.7.2 Suggested first log entry content

If you want a concrete model, the first real entry can look like this:

Research Log: 2026-04-14

Objective

Create a reproducible research environment for the battery simulation
 ↳ companion manual.

Environment

- Active Python environment: ``sib-research``
- MATLAB version: ``R2025b``
- Git commit: not initialized yet
- Dataset or paper in use: none

Work Performed

Installed the pinned Python stack from ``environment.yml``, registered the
 ↳ Jupyter kernel, and created the base repository scaffold using
 ↳ ``src/bootstrap_project.py``. Verified that imports for NumPy, SciPy,
 ↳ Pandas, and Matplotlib succeed. Opened MATLAB and confirmed that Simulink
 ↳ and Simscape are available, but still need to confirm Simscape Battery.

Observations

Python executable points into the conda environment as expected. Jupyter sees
 ↳ the ``Python (sib-research)`` kernel. MATLAB launches normally. No datasets
 ↳ downloaded yet.

Decisions and Rationale

Chose ``conda`` over ``venv`` because later chapters will use compiled scientific
 ↳ libraries and cross-platform reproducibility is more important than
 ↳ minimal tooling. Chose to keep raw and processed data in separate
 ↳ directories from day one to avoid later provenance confusion.

Problems Encountered

``conda activate`` did not work until running ``conda init bash`` and restarting
 ↳ the terminal.

Next Steps

1. Initialize Git and commit the scaffold.
2. Write Python and MATLAB hello-battery sanity checks.
3. Confirm Simscape Battery availability.

1.7.3 Walkthrough 3 code explanation

The script creates a dated Markdown file rather than a notebook, Word document, or proprietary note format. That choice is deliberate. Markdown is plain text, easy to diff, easy to search, and comfortable inside Git repositories.

The filename pattern YYYY-MM-DD_setup.md sorts naturally in chronological order. This sounds trivial until a project accumulates dozens of entries. Human-friendly ordering becomes a major quality-of-life feature.

The template sections are short because a research log should be sustainable. If the template is too long, you will stop using it. The fields here capture the essentials: objective, environment, work performed, observations, decisions, problems, and next steps.

The script refuses to overwrite an existing entry for the day. That is a good safety behavior. You can always edit the existing log, but you should not accidentally erase it.

1.7.4 Walkthrough 3 expected output

The terminal should show something like:

Created log entry: logs/2026-04-14_setup.md

The log file should exist and open as plain text. Once edited, it should read like a technical lab notebook rather than a diary. The difference matters. “Spent a while trying things” is not useful. “Tried conda activate before shell initialization; fixed with conda init bash and terminal restart” is useful.

1.7.5 What belongs in the log and what does not

A good rule is this: record anything that explains a result, a failure, or a future decision.

Record:

- environment changes
- dataset sources and licenses
- solver choices
- smoothing choices
- reasons for discarding or keeping data segments
- exact paper figures you are targeting in a reproduction
- uncertainties and ambiguities

Do not record:

- every command you typed if it had no consequence
- vague emotional summaries without technical content
- outputs that are already reproducible and fully captured elsewhere unless they matter interpretively

1.7.6 Walkthrough 3 troubleshooting

1. You create the template but never update it
Symptom: the repository has empty log files.
Fix: treat the generated template as a draft to be filled immediately, not later.
2. The log becomes a second README
Symptom: every entry repeats project background instead of recording decisions.
Fix: focus on what changed today.
3. You only log successes
Symptom: later you cannot reconstruct which dead ends you already tried.
Fix: record failed preprocessing or modeling attempts briefly and honestly.

1.7.7 Walkthrough 3 reflection

This exercise taught the human side of reproducibility. Later, when we compare model fidelities or troubleshoot estimation drift, your log will often be the only place where the rationale survives.

1.8 Guided Walkthrough 4: Initialize Git and Build a Commit History Worth Keeping

Learning objective: Turn the scaffold into a versioned research repository, add a battery-research-appropriate `.gitignore`, and make the first sequence of meaningful commits.

Git becomes helpful only when used before the repository is messy. Starting version control after a month of experiments is like deciding to label sample vials after mixing half of them together. We are going to initialize the repository now and establish a commit style that later supports actual research.

1.8.1 Walkthrough 4 code

Create `.gitignore` in the repository root:

```
# Python cache and notebook state
__pycache__/
*.py[cod]
.ipynb_checkpoints/

# Virtual environments
.venv/
env/
ENV/

# Tool caches
.pytest_cache/
.mypy_cache/

# OS clutter
.DS_Store
Thumbs.db
```

```
# Large local data: keep directory placeholders, ignore contents
data/raw/*
!data/raw/.gitkeep
data/processed/*
!data/processed/.gitkeep
```

```
# Temporary outputs
results/tmp/
figures/tmp/
```

```
# MATLAB generated files
*.asv
*.slxc
*.mexw64
*.mexa64
*.mexmaci64
```

Initialize the repository and make the first commits:

```
git init
git add environment.yml
git add requirements-venv.txt
git add src/bootstrap_project.py
git add src/env_report.py
git add src/create_log_entry.py
git add README.md
git add .gitignore
git add data/raw/.gitkeep
git add data/processed/.gitkeep
git add notebooks/.gitkeep
git add results/.gitkeep
git add figures/.gitkeep
git add papers/.gitkeep
git add logs/2026-04-14_setup.md
git add matlab/.gitkeep
git commit -m "Initialize reproducible battery research scaffold"
```

Now generate an environment report so the repository captures a first machine-readable environment artifact:

```
python src/env_report.py
git add results/environment_report.json
git commit -m "Add machine-readable environment report"
```

Register the current default branch explicitly:

```
git branch -M main
```

If you already created a GitHub repository in the browser, connect it:

```
git remote add origin
↪ https://github.com/YOUR_USERNAME/sib-research-companion.git
git push -u origin main
```

1.8.2 A meaningful 10-commit sequence

Your chapter deliverable asks for at least ten meaningful commits. Here is a clean sequence you can actually use over the rest of this chapter:

1. Initialize reproducible battery research scaffold
2. Add machine-readable environment report
3. Add MATLAB toolbox verification script
4. Add Python hello-battery sanity check
5. Add MATLAB hello-battery sanity check
6. Save Python sanity-check outputs
7. Save MATLAB sanity-check outputs
8. Refine README with setup and workflow notes
9. Add research-log entry for chapter 1
10. Document chapter 1 deliverable completion

These are meaningful because each one captures a coherent step, not because the messages sound formal.

1.8.3 Walkthrough 4 code explanation

The `.gitignore` is tuned for research. It does not blindly ignore results/ and figures/ wholesale, because some results are worth tracking. Instead, it ignores only explicitly temporary subdirectories if you create them later. That choice teaches an important distinction: some generated artifacts are disposable, while some are central to the research record.

We ignore contents of `data/raw/` and `data/processed/` by default because public battery datasets can be large, and many are better downloaded by script or documented URL than committed to Git. The `.gitkeep` exceptions preserve the folder structure in the repository. This is standard practice.

The first commit gathers the reproducibility scaffold into a single checkpoint. The second commit adds the environment report as an artifact. Notice the pattern: one commit establishes capability, the next records evidence that the capability works. We will repeat that pattern throughout the manual.

1.8.4 Walkthrough 4 expected output

After the first commit, `git status` should say:

```
On branch main
nothing to commit, working tree clean
```

If you run:

```
git log --online --decorate --graph -n 5
```

you should see a short history of your recent commits, for example:

```
* a1b2c3d (HEAD -> main) Add machine-readable environment report
* e4f5g6h Initialize reproducible battery research scaffold
```

Your commit hashes will differ, of course.

1.8.5 What makes a commit “meaningful” in research

A meaningful research commit is not necessarily a large one. It is one whose purpose you can summarize in one sentence.

Good:

- Add parser for CALCE HPPC current and voltage columns
- Fix sign convention in UDDS current import
- Compare DFN and SPMe runtime on same experiment
- Add EKF tuning sweep notebook

Poor:

- updates
- misc
- stuff
- fixes

The point is not style policing. It is traceability. Six weeks later, when a result changes, a meaningful commit history lets you find why.

1.8.6 Walkthrough 4 troubleshooting

1. `git commit` refuses because user name or email is not configured
 Symptom: Git prompts you to set identity.
 Fix: run the `git config --global user.name` and `user.email` commands from the setup section.
2. You accidentally commit a large dataset
 Symptom: `git status` shows many files inside `data/raw/`.
 Fix: check that `.gitignore` exists before `git add .`, and use `git rm --cached` on accidentally tracked data files.
3. Jupyter notebooks create noisy diffs
 Symptom: commits are dominated by output-cell changes.
 Fix: keep exploratory notebooks, but move stable logic into scripts under `src/`.

1.8.7 Walkthrough 4 reflection

This exercise taught that version control is part of scientific method, not just collaboration etiquette. Later, when reproducing papers or tuning estimators, your commit history will become one of the main tools for diagnosing divergence.

1.9 Guided Walkthrough 5: Run a “Hello Battery” Sanity Check in Python and MATLAB

Learning objective: Verify that both ecosystems can produce a simple, battery-flavored result with known behavior, known units, and stored outputs.

Now we do the first genuinely battery-shaped computation of the manual. It is still intentionally simple. We are not building a real cell model yet. We are building a sanity check that proves the toolchain can generate, save, and interpret a small discharge experiment in both Python and MATLAB.

We will use the following pedagogical model:

$$\text{SOC}(t) = \text{SOC}_0 - \frac{It}{3600Q_{\text{nom}}} \quad (1.1)$$

$$V_{\text{OCV}}(\text{SOC}) = 3.0 + 0.6 \text{ SOC} \quad (1.2)$$

$$V_{\text{term}}(t) = V_{\text{OCV}}(\text{SOC}(t)) - IR_0 \quad (1.3)$$

This is not a publishable model. It is a teaching model. The goal is to verify the workflow, not to represent a chemistry accurately. The model is intentionally transparent so that you can predict what the output should look like before you run it.

We choose:

- nominal capacity: $Q_{\text{nom}} = 2.0 \text{ Ah}$
- current: $I = 2.0 \text{ A}$, which is a 1C discharge
- ohmic resistance: $R_0 = 0.05 \Omega$
- initial SOC: $\text{SOC}_0 = 1.0$
- simulation time: $t_{\text{final}} = 3600 \text{ s}$

From Equation (1.1), a 1C discharge over 3600 s should take SOC from 1.0 to 0.0. From Equations (1.2) and (1.3), OCV should fall linearly from 3.6 V to 3.0 V, while terminal voltage should sit 0.1 V lower, from 3.5 V to 2.9 V. If your output does not look like that, the toolchain or the code is wrong.

1.9.1 Part A: Python sanity check

Create src/hello_battery.py:

```
from __future__ import annotations

from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
def main() -> None:
    # Create output directories if they do not already exist.
    Path("results").mkdir(parents=True, exist_ok=True)
    Path("figures").mkdir(parents=True, exist_ok=True)

    # Define a simple teaching model for a 2 Ah cell discharged at 1C.
    nominal_capacity_ah = 2.0
    discharge_current_a = 2.0
    ohmic_resistance_ohm = 0.05
    initial_soc = 1.0
    time_step_s = 1.0
    final_time_s = 3600.0

    # Build the simulation time vector.
    time_s = np.arange(0.0, final_time_s + time_step_s, time_step_s)

    # Coulomb counting: SOC falls linearly under constant current discharge.
```



```

soc = initial_soc - (
    discharge_current_a * time_s / (3600.0 * nominal_capacity_ah)
)
soc = np.clip(soc, 0.0, 1.0)

# Simple linear OCV curve used only for a sanity check.
ocv_v = 3.0 + 0.6 * soc

# Terminal voltage includes an ohmic drop during discharge.
terminal_voltage_v = ocv_v - discharge_current_a * ohmic_resistance_ohm

# Instantaneous power delivered to the load.
power_w = terminal_voltage_v * discharge_current_a

# Compute delivered energy by integrating power over time.
delivered_energy_wh = np.trapz(power_w, time_s) / 3600.0

# Store results in a tidy table.
results_table = pd.DataFrame(
    {
        "time_s": time_s,
        "soc_fraction": soc,
        "ocv_v": ocv_v,
        "terminal_voltage_v": terminal_voltage_v,
        "current_a": discharge_current_a,
        "power_w": power_w,
    }
)

results_csv_path = Path("results") / "hello_battery_python.csv"
results_table.to_csv(results_csv_path, index=False)

# Create a two-panel figure.
fig, axes = plt.subplots(2, 1, figsize=(8, 8), sharex=True)

axes[0].plot(time_s, soc, linewidth=2, color="tab:blue")
axes[0].set_ylabel("SOC [-]")
axes[0].set_title("Hello Battery Sanity Check (Python)")
axes[0].grid(True, alpha=0.3)
axes[0].set_ylim(-0.05, 1.05)

axes[1].plot(time_s, ocv_v, linewidth=2, label="OCV", color="tab:green")
axes[1].plot(
    time_s,
    terminal_voltage_v,
    linewidth=2,
    label="Terminal voltage",
    color="tab:red",
)
axes[1].set_xlabel("Time [s]")
axes[1].set_ylabel("Voltage [V]")
axes[1].grid(True, alpha=0.3)
axes[1].legend()

```

```

figure_path = Path("figures") / "hello_battery_python.png"
fig.tight_layout()
fig.savefig(figure_path, dpi=200)
plt.close(fig)

print(f"Saved results table to: {results_csv_path}")
print(f"Saved figure to: {figure_path}")
print(f"Initial terminal voltage: {terminal_voltage_v[0]:.2f} V")
print(f"Final terminal voltage: {terminal_voltage_v[-1]:.2f} V")
print(f"Delivered capacity: {discharge_current_a * final_time_s /
    ↪ 3600.0:.2f} Ah")
print(f"Delivered energy: {delivered_energy_wh:.2f} Wh")
if __name__ == "__main__":
    main()

```

Run it:

```
python src/hello_battery.py
```

1.9.1.1 Python sanity check explanation The first two `Path(...).mkdir(...)` calls make the script robust. A reproducible script should not assume that output directories already exist.

The parameter block is deliberately explicit. We are not hiding constants in cryptic variable names. This is good scientific style: a reader should be able to identify capacity, current, resistance, initial SOC, time step, and simulation horizon immediately.

`np.arange(0.0, final_time_s + time_step_s, time_step_s)` includes the final time point. If we omitted the `+ time_step_s`, the last sample at 3600 s would be missing. That sounds minor, but end-point consistency becomes important later when comparing outputs across tools.

The SOC calculation is just Equation (1.1). We then clip with `np.clip` so the value cannot drop below zero. For this chosen current and duration, the unclipped result lands exactly at zero, but clipping is still a healthy protective pattern.

The OCV line implements Equation (1.2). Again, this is not a chemistry model. It is an intentionally transparent surrogate so that the expected output is obvious.

The terminal voltage subtracts the constant ohmic drop, which is $IR_0 = 2.0 \times 0.05 = 0.1$ V. Therefore, the terminal curve should be parallel to the OCV curve and always 0.1 V lower. If it is not, the code or sign convention is wrong.

`np.trapz(power_w, time_s) / 3600.0` numerically integrates power in watts over time in seconds, then divides by 3600 to convert joules-per-second integrated over seconds into watt-hours. This is a good early reminder that unit discipline matters everywhere.

We store results as a tidy `pandas.DataFrame` because later chapters will rely heavily on tabular outputs. Even this toy script teaches the habit that plots and machine-readable tables should both be saved.

The plot is split into two panels because SOC and voltage live on different scales and tell different stories. Good plotting is part of research competence, not decoration.

1.9.1.2 Python sanity check expected output

In the terminal, you should see:

```
Saved results table to: results/hello_battery_python.csv
Saved figure to: figures/hello_battery_python.png
Initial terminal voltage: 3.50 V
Final terminal voltage: 2.90 V
Delivered capacity: 2.00 Ah
Delivered energy: 6.40 Wh
```

The CSV should contain 3601 rows, one for each second from 0 to 3600 inclusive.

The figure should have two panels.

The top panel, SOC vs time, should be a perfectly straight descending line from 1.0 at 0 s to 0.0 at 3600 s. If it curves, steps, or ends somewhere other than zero, something is off.

The bottom panel should show two straight lines. The green OCV line should descend from 3.6 V to 3.0 V. The red terminal-voltage line should descend from 3.5 V to 2.9 V and remain exactly 0.1 V below the OCV line for the entire discharge. That constant gap is the visual signature of the ohmic term in Equation (1.3).

A wrong result often looks like one of three things: the terminal voltage is above OCV, which indicates a sign error; the final SOC is negative, which indicates missing clipping or a duration mismatch; or the final voltage is not 2.9 V, which indicates incorrect current, capacity, or resistance.

1.9.1.3 Python sanity check troubleshooting

1. The script runs but no figure appears on screen
Symptom: nothing pops up visually.
Fix: this script saves the figure to disk and closes it intentionally. Check figures/hello_battery_python.png.
2. ImportError: No module named 'matplotlib' or pandas
Symptom: imports fail even though the environment exists.
Fix: confirm the environment is activated before running the script.
3. Delivered energy is not approximately 6.40 Wh
Symptom: printed energy is substantially different.
Fix: check the unit conversion in the integration step and confirm the current is 2.0 A.

1.9.1.4 Python sanity check reflection This Python exercise taught a complete mini-workflow: define parameters, compute derived quantities, save a table, save a figure, and print interpretable summary metrics. That pattern will return in nearly every later chapter.

1.9.2 Part B: MATLAB toolbox verification and sanity check

Before we run the MATLAB battery check, verify the products we need later.

Create matlab/check_toolboxes.m:

```
requiredProducts = [
    "MATLAB"
    "Simulink"
```

```

    "Simscape"
    "Simscape Battery"
    "Control System Toolbox"
    "Optimization Toolbox"
1;

installedProducts = ver;
installedNames = string({installedProducts.Name});

fprintf("MATLAB product check\n");
fprintf("-----\n");

for k = 1:numel(requiredProducts)
    isInstalled = any(installedNames == requiredProducts(k));
    if isInstalled
        statusText = "FOUND";
    else
        statusText = "MISSING";
    end
    fprintf("%-24s : %s\n", requiredProducts(k), statusText);
end

```

Run it in MATLAB:

```
check_toolboxes
```

Expected output is a short table of FOUND and MISSING statuses. If Simscape Battery is missing, note that explicitly in your research log.

Now create matlab/hello_battery.m:

```

clear;
clc;
close all;

if ~exist("results", "dir")
    mkdir("results");
end

if ~exist("figures", "dir")
    mkdir("figures");
end

nominalCapacityAh = 2.0;
dischargeCurrentA = 2.0;
ohmicResistanceOhm = 0.05;
initialSoc = 1.0;
timeStepS = 1.0;
finalTimeS = 3600.0;

timeS = (0:timeStepS:finalTimeS)';
soc = initialSoc - (dischargeCurrentA .* timeS) ./ (3600.0 .*
    ↪ nominalCapacityAh);
soc = max(0.0, min(1.0, soc));

ocvV = 3.0 + 0.6 .* soc;

```

```

terminalVoltageV = ocvV - dischargeCurrentA .* ohmicResistanceOhm;
powerW = terminalVoltageV .* dischargeCurrentA;

deliveredEnergyWh = trapz(timeS, powerW) / 3600.0;

resultsTable = table( ...
    timeS, ...
    soc, ...
    ocvV, ...
    terminalVoltageV, ...
    repmat(dischargeCurrentA, size(timeS)), ...
    powerW, ...
    'VariableNames', ...
    {'time_s', 'soc_fraction', 'ocv_v', 'terminal_voltage_v', 'current_a',
    ↪ 'power_w'} ...
);

writetable(resultsTable, fullfile("results", "hello_battery_matlab.csv"));

figureHandle = figure("Position", [100, 100, 800, 700]);

subplot(2, 1, 1);
plot(timeS, soc, "LineWidth", 2, "Color", [0.0, 0.45, 0.74]);
grid on;
ylabel("SOC [-]");
title("Hello Battery Sanity Check (MATLAB)");
ylim([-0.05, 1.05]);

subplot(2, 1, 2);
plot(timeS, ocvV, "LineWidth", 2, "Color", [0.47, 0.67, 0.19]);
hold on;
plot(timeS, terminalVoltageV, "LineWidth", 2, "Color", [0.85, 0.33, 0.10]);
grid on;
xlabel("Time [s]");
ylabel("Voltage [V]");
legend("OCV", "Terminal voltage", "Location", "best");

exportgraphics(figureHandle, fullfile("figures", "hello_battery_matlab.png"),
    ↪ "Resolution", 200);

fprintf("Saved results table to: %s\n", fullfile("results",
    ↪ "hello_battery_matlab.csv"));
fprintf("Saved figure to: %s\n", fullfile("figures",
    ↪ "hello_battery_matlab.png"));
fprintf("Initial terminal voltage: %.2f V\n", terminalVoltageV(1));
fprintf("Final terminal voltage: %.2f V\n", terminalVoltageV(end));
fprintf("Delivered capacity: %.2f Ah\n", dischargeCurrentA * finalTimeS /
    ↪ 3600.0);
fprintf("Delivered energy: %.2f Wh\n", deliveredEnergyWh);

```

Run it in MATLAB:

```
hello_battery
```

1.9.2.1 MATLAB sanity check explanation The structure mirrors the Python version on purpose. That parallelism is pedagogically useful. Later, when we move data between PyBaMM and MATLAB, you want the two ecosystems to feel different in syntax but aligned in workflow.

The `mkdir` guards are the MATLAB equivalent of the Python directory-creation calls. Again, robust scripts should not assume a preexisting folder state.

The time vector is created as a column vector with `(0:timeStepS:finalTimeS)'`. The transpose matters because MATLAB tables and plotting functions often behave more predictably when vectors are column-oriented.

The SOC computation uses elementwise operators `.*` and `./`. This is one of the biggest MATLAB habits to internalize early. If you accidentally use matrix operators here, the script will either fail or behave incorrectly.

The clamp `max(0.0, min(1.0, soc))` is the MATLAB analogue of `np.clip`.

`table(...)` creates a labeled table rather than a bare matrix. As in Python, storing named columns is far better research practice than saving anonymous arrays and hoping you remember what each column meant.

`exportgraphics` is preferred over old screenshot-style save behavior because it gives cleaner, reproducible figure exports.

1.9.2.2 MATLAB sanity check expected output The MATLAB command window should print:

```
Saved results table to: results\hello_battery_matlab.csv
Saved figure to: figures\hello_battery_matlab.png
Initial terminal voltage: 3.50 V
Final terminal voltage: 2.90 V
Delivered capacity: 2.00 Ah
Delivered energy: 6.40 Wh
```

The slash direction in file paths may differ by operating system.

The plot should match the Python result qualitatively and numerically. The top panel should be a straight-line SOC decline from 1 to 0. The bottom panel should show OCV and terminal-voltage lines with a constant 0.1 V gap.

If the MATLAB and Python outputs disagree materially, stop and resolve that now. Cross-tool agreement on simple cases is the first step toward trusting more complex models later.

1.9.2.3 MATLAB sanity check troubleshooting

1. MATLAB cannot find the script

Symptom: Undefined function or variable 'hello_battery'.

Fix: make sure the current MATLAB folder is the repository root or add the `matlab/` folder to the path.

2. Simscape Battery shows as missing

Symptom: the product check reports MISSING.

Fix: note it in your log. This does not block this chapter, but it will matter later.

3. The CSV is saved but the figure export fails

Symptom: `exportgraphics` errors.

Fix: ensure you are using R2024b or later; on older versions, use `saveas` as a fallback, but note the deviation in your log.

1.9.2.4 MATLAB sanity check reflection This MATLAB exercise taught the same research pattern in a second ecosystem. That is exactly the point. Publishable battery work often spans tools. Consistent habits across tools matter more than memorizing one syntax perfectly.

1.10 Open-Ended Exercises

These exercises ask you to modify the guided work rather than simply rerun it. Try them before reading the worked solutions.

1.10.1 Exercise 1

Modify the Python `hello_battery.py` script so it runs three discharges at 0.5C, 1C, and 2C for the same 2 Ah cell, then saves one combined CSV with a `c_rate` column.

Hints:

At 0.5C, the current should be 1 A. At 2C, the current should be 4 A. You will need a loop over current values and a way to concatenate several tables together.

1.10.2 Exercise 2

Extend `src/env_report.py` so it also records the SHA-256 hash of `environment.yml`.

Hints:

Use Python's `hashlib` module. The point is to capture not only installed-package versions but also the exact environment-spec file that was intended.

1.10.3 Exercise 3

Create a second log-template generator that makes entries named `YYYY-MM-DD_experiment.md` instead of `YYYY-MM-DD_setup.md`, and add a section called Planned Figure or Table.

Hints:

Reuse `src/create_log_entry.py` rather than writing from scratch. Change only what actually differs.

1.11 What Changes for Sodium-Ion?

At this early stage, the main sodium-ion adaptation is not a different equation set. It is a different standard of bookkeeping.

Sodium-ion work often depends on sparser public data, less standardized parameter releases, and more figure digitization than mainstream lithium-ion work. That means provenance matters even more. In a lithium-ion project, you may be able to rely on a well-known public dataset and a widely reused parameter set. In a sodium-ion project, you may need to record exactly which review paper you extracted an OCV shape from, which chemistry family the cell belongs to, whether the hard-carbon behavior reflects the slope region or plateau region, and how you converted a published current density into an absolute current for your assumed cell area or capacity.

So, from the very first chapter, get into the habit of tagging artifacts with chemistry context. A filename like `ocv_curve.csv` is weak. A filename like `hard_carbon_p2_naion_ocv_digitized_2026-04-14.csv` is stronger. A log entry that says “used SIB data” is weak. One that says “used digitized hard-carbon half-cell OCV from [paper], Figure 2b, units converted from mAh g⁻¹ to SOC using author-reported reversible capacity” is much stronger.

That discipline will matter more and more as the manual progresses.

1.12 Worked Solutions to the Open-Ended Exercises

1.12.1 Solution to Exercise 1

A clean way to solve the multi-C-rate problem is to wrap the discharge logic in a function and concatenate results.

```
from __future__ import annotations

from pathlib import Path

import numpy as np
import pandas as pd

Path("results").mkdir(parents=True, exist_ok=True)

nominal_capacity_ah = 2.0
ohmic_resistance_ohm = 0.05
initial_soc = 1.0
time_step_s = 1.0

c_rates = [0.5, 1.0, 2.0]
all_results = []

for c_rate in c_rates:
    discharge_current_a = c_rate * nominal_capacity_ah
    final_time_s = 3600.0 / c_rate
    time_s = np.arange(0.0, final_time_s + time_step_s, time_step_s)

    soc = initial_soc - (
        discharge_current_a * time_s / (3600.0 * nominal_capacity_ah)
    )
    soc = np.clip(soc, 0.0, 1.0)

    ocv_v = 3.0 + 0.6 * soc
    terminal_voltage_v = ocv_v - discharge_current_a * ohmic_resistance_ohm

    one_case = pd.DataFrame(
        {
            "c_rate": c_rate,
            "time_s": time_s,
            "soc_fraction": soc,
            "ocv_v": ocv_v,
            "terminal_voltage_v": terminal_voltage_v,
            "current_a": discharge_current_a,
```



```

    }
)
all_results.append(one_case)

combined_results = pd.concat(all_results, ignore_index=True)
combined_results.to_csv("results/hello_battery_multi_rate.csv", index=False)

print(combined_results.groupby("c_rate")["terminal_voltage_v"].agg(["first",
↪ "last"]))

```

The key idea is that current scales with C-rate through $I = C_{\text{rate}}Q_{\text{nom}}$, and full-discharge time scales inversely. The expected pattern is that higher C-rate gives a larger ohmic drop and shorter discharge time. With this teaching model, the 2C case starts 0.2 V below OCV because $4 \text{ A} \times 0.05 \Omega = 0.2 \text{ V}$.

1.12.2 Solution to Exercise 2

Here is a version of the environment report with a SHA-256 hash for environment.yml.

```

from __future__ import annotations

import hashlib
import json
import platform
import subprocess
import sys
from datetime import datetime, timezone
from importlib.metadata import PackageNotFoundError, version
from pathlib import Path

PACKAGES_TO_REPORT = [
    "numpy",
    "scipy",
    "pandas",
    "matplotlib",
    "jupyterlab",
    "ipykernel",
]

def package_version(package_name: str) -> str:
    try:
        return version(package_name)
    except PackageNotFoundError:
        return "NOT_INSTALLED"

def sha256_of_file(file_path: Path) -> str:
    return hashlib.sha256(file_path.read_bytes()).hexdigest()

def git_commit_hash() -> str:
    try:
        return subprocess.check_output(
            ["git", "rev-parse", "HEAD"],
            text=True,
        ).strip()
    except Exception:
        return "NOT_A_GIT_REPOSITORY"

```

```

def main() -> None:
    results_dir = Path("results")
    results_dir.mkdir(parents=True, exist_ok=True)

    environment_file = Path("environment.yml")

    report = {
        "generated_utc": datetime.now(timezone.utc).isoformat(),
        "python_executable": sys.executable,
        "python_version": platform.python_version(),
        "platform": platform.platform(),
        "git_commit": git_commit_hash(),
        "environment_yaml_sha256": sha256_of_file(environment_file)
        if environment_file.exists()
        else "FILE_NOT_FOUND",
        "packages": {
            package_name: package_version(package_name)
            for package_name in PACKAGES_TO_REPORT
        },
    }

    output_path = results_dir / "environment_report.json"
    output_path.write_text(json.dumps(report, indent=2), encoding="utf-8")
    print(json.dumps(report, indent=2))
if __name__ == "__main__":
    main()

```

The research value here is subtle but real. Two environments can sometimes report similar installed packages while differing in the intended spec file. Hashing `environment.yml` gives you an additional integrity check.

1.12.3 Solution to Exercise 3

A minimal adaptation of the log generator is enough.

```
from __future__ import annotations
```

```
from datetime import datetime
```

```
from pathlib import Path
```

```
TEMPLATE = """# Research Log: {date}
```

```
## Objective
```

```
Write one sentence describing today's concrete objective.
```

```
## Planned Figure or Table
```

```
Describe the exact figure, table, or output artifact you intend to produce.
```

```
## Environment
```

- Active Python environment:
- MATLAB version:

- Git commit:
- Dataset or paper in use:

Work Performed

Describe what you actually did.

Observations

Record numerical outputs, runtime notes, and figure behavior.

Decisions and Rationale

Explain why you made methodological choices.

Problems Encountered

Include exact failure modes and error messages where useful.

Next Steps

Write the next one to three actions.

"""

```
def main() -> None:
    logs_dir = Path("logs")
    logs_dir.mkdir(parents=True, exist_ok=True)

    date_string = datetime.now().strftime("%Y-%m-%d")
    output_path = logs_dir / f"{date_string}_experiment.md"

    if output_path.exists():
        print(f"Log entry already exists: {output_path}")
        return

    output_path.write_text(TEMPLATE.format(date=date_string), encoding="utf-8")
    print(f"Created experiment log entry: {output_path}")
if __name__ == "__main__":
    main()
```

This version is useful once the project shifts from environment setup into actual model runs and figure generation. The new Planned Figure or Table section encourages intentional work rather than random exploration.

1.13 Chapter Summary and Skill Checklist

You should now have these skills in working memory:

- create and activate an isolated Python environment
- verify package versions instead of assuming them
- register and select the correct Jupyter kernel
- verify MATLAB product availability for later battery workflows
- initialize a Git repository early
- write a research-appropriate .gitignore

- maintain a dated Markdown research log
- separate raw data, processed data, notebooks, reusable code, and generated outputs
- run a small battery-flavored sanity check in both Python and MATLAB
- save both machine-readable outputs and figure files

You should now be able to answer “yes” to all of the following:

- I can create a reproducible Python environment from a file rather than from memory.
- I can prove which interpreter and package versions generated an output.
- I have a repository structure that distinguishes source code from artifacts.
- I can explain why `data/raw/` and `data/processed/` must remain separate.
- I have a research-log habit that records decisions, not just successes.
- I can initialize Git, make meaningful commits, and avoid committing bulky raw data by accident.
- I can run and interpret a simple battery sanity check in both Python and MATLAB.
- I know whether my MATLAB installation includes the toolboxes needed for later chapters.

If any of those are still shaky, revisit the corresponding walkthrough now. Later chapters assume them.

1.14 Deliverable

The deliverable from your plan is a GitHub repository with a working environment, a research log template, and a passing sanity check in both Python and MATLAB, with a commit history showing at least ten meaningful commits.

Approach it as a mini research artifact, not a homework submission. Your repository should contain `environment.yml`, the scaffold scripts, the log template script, the `.gitignore`, both sanity-check scripts, the resulting CSV and PNG outputs, and a short README explaining how to rerun everything. Your Git history should tell the story of the setup in coherent steps rather than one giant dump commit.

A good completion state looks like this:

- `README.md` explains the project purpose and setup commands.
- `environment.yml` is present and matches the actual working environment.
- `results/environment_report.json` exists.
- `logs/YYYY-MM-DD_setup.md` contains real content, not placeholders.
- `results/hello_battery_python.csv` and `figures/hello_battery_python.png` exist.
- `results/hello_battery_matlab.csv` and `figures/hello_battery_matlab.png` exist.
- `git log --oneline -n 10` shows a sensible progression of work.

If you want a good final commit message for this chapter, use something like:

Complete chapter 1 research environment deliverable

Then add a final log entry summarizing what is verified, what remains uncertain, and what Chapter 2 will require.

1.15 Further Practice and Reading

Start with one or two reproducibility papers, one or two official docs, and one or two community resources you will actually revisit.

- Sandve, Nekrutenko, Taylor, and Hovig, “Ten Simple Rules for Reproducible Computational Research.” A concise paper worth internalizing early.
- Wilson et al., “Best Practices for Scientific Computing.” Older, still foundational, and directly relevant to the habits in this chapter.
- JupyterLab documentation: bookmark the user guide and kernel-management sections.
- Conda or Miniforge documentation: especially environment creation and export commands.
- MathWorks documentation for ver, MATLAB Projects, and later Simscape Battery.
- Pro Git by Scott Chacon and Ben Straub: still the cleanest Git reference for researchers.
- Community resources: Jupyter Discourse, Scientific Python Forum, MATLAB Central.

Lab Chapter 2: *Scientific Python Refresher for Battery Work* is next.